

Algoritmos e Lógica de Programação

Análise e Desenvolvimento de Sistemas (ADS)

2º Semestre de 2025

Prof. Marcelo Amorim



Professor Marcelo Amorim

Formação

Mestre e Doutor pelo IME-USP.

Experiência

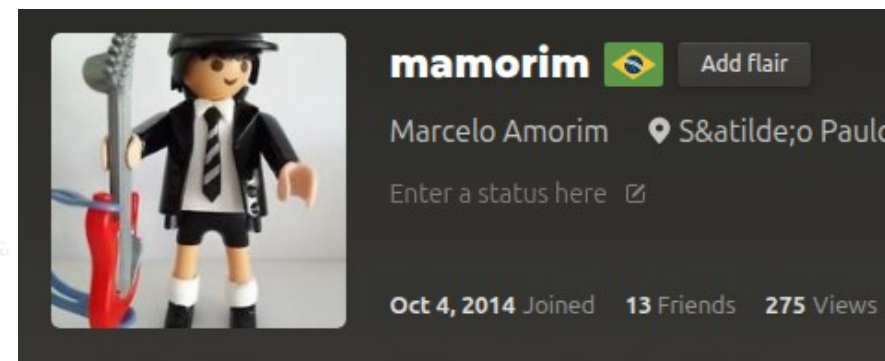
Professor universitário desde 2001. Já atuou como coordenador de cursos e gestor acadêmico em uma holding nacional de educação, além de consultor e desenvolvedor de software. Atua principalmente com Teoria da Computação e desenvolvimento de software, com especialização em Web/Mobile.

Contato

marcelo.amorim@fecap.edu

Quem me desafia numa partida de xadrez?

chess.com/member/mamorim



Por que estudar Algoritmos e Lógica de Programação?

- É a base de tudo que vem depois no curso: toda linguagem e tecnologia que você vai usar depende de raciocínio lógico bem construído.
- O objetivo não é decorar sintaxe, e sim desenvolver a capacidade de resolver problemas passo a passo, de forma estruturada.
- Sem lógica sólida, nenhuma linguagem de programação “salva” o profissional — essa é a habilidade que fica.
- É o primeiro passo de uma trilha: aqui você constrói a base para as próximas disciplinas de programação do curso.

O que vamos aprender

Fundamentos

Lógica, algoritmos, variáveis, tipos de dados e expressões aritméticas, relacionais e lógicas.

Estruturas de Controle

Estrutura sequencial, decisão (simples, composta, encadeada, múltipla escolha) e repetição (enquanto, faça/enquanto, para).

Dados Estruturados

Vetores, matrizes, registros e persistência de dados em arquivos.

Modularização

Funções e procedimentos, passagem de parâmetros (por valor e por referência) e recursividade.



Como vamos trabalhar

Metodologia

- Aulas expositivas com aplicação prática de exercícios e exemplos contextualizados.
- Integração com o Projeto Interdisciplinar (PI) do semestre, de caráter extensionista.
- Avaliação combina Prova Intermediária + Atividades, PI e Prova Final.

Ferramentas

- Linguagem: C#, com a plataforma .NET.
- Editor de código: Visual Studio Code.
- Os códigos das aulas ficarão disponíveis em um repositório no GitHub, cujo link será compartilhado no início do curso.



O que é um algoritmo?

Lembra da atividade de instruir alguém a dobrar o papel? A ambiguidade que apareceu ali é exatamente o problema que um algoritmo resolve.

Definição

Algoritmo é uma sequência finita de passos, bem definidos e ordenados, que leva à solução de um problema ou à realização de uma tarefa.

Para funcionar, um algoritmo precisa ser:

- Finito — tem começo, meio e fim, não roda para sempre.
- Preciso — cada passo é claro, sem margem para interpretação dupla.
- Ordenado — a sequência dos passos importa.
- Executável — cada passo pode de fato ser realizado.

Lógica e processamento de dados

Lógica

É o raciocínio estruturado que usamos para chegar a uma conclusão ou resolver um problema — organizar o pensamento antes de organizar o código.

Em programação, a lógica é o que garante que o algoritmo realmente faz o que deveria fazer.

Processamento de dados

Todo processamento segue o mesmo ciclo básico:

- Entrada — os dados que o programa recebe.
- Processamento — o que é feito com esses dados.
- Saída — o resultado entregue ao usuário.

Algoritmo x Programa x Linguagem de programação

- **Algoritmo** é a ideia — a sequência de passos que resolve o problema, independente de qualquer linguagem.
- **Programa** é a implementação dessa ideia em código, escrita numa linguagem específica para ser executada pelo computador.
- **Linguagem de programação** é a ferramenta e a notação usadas para escrever o programa — C#, Python, Java, etc.

Uma analogia

O algoritmo é a receita em si (os passos). O programa é a receita escrita à mão. A linguagem de programação é o idioma em que você escreveu essa receita.

Exemplos de algoritmos do dia a dia

Receita culinária

Uma sequência de passos ordenados (ingredientes, modo de preparo) que leva a um prato pronto.

Manual de montagem

Passos numerados para montar um móvel, sempre na mesma ordem, sem pular etapas.

Rota do GPS

Uma sequência de instruções de navegação que leva do ponto A ao ponto B.

Troca de pneu

Passos precisos e ordenados — errar a ordem pode até ser perigoso.



Como construir um algoritmo narrativo

Algoritmo narrativo é escrever os passos em português claro (ou pseudocódigo), antes de qualquer linguagem de programação. É o rascunho da solução.

Boas práticas:

- Um passo por linha, numerado.
- Comece cada passo com um verbo de ação (leia, calcule, exiba, verifique...).
- Siga a ordem exata em que as coisas devem acontecer.
- Evite ambiguidade — se dá para interpretar de duas formas, reescreva.
- Pense em entrada, processamento e saída, na ordem em que aparecem.

Exemplo prático: algoritmo narrativo

Problema: calcular a média de duas notas e informar se o aluno foi aprovado (média ≥ 6).

1. Leia a primeira nota do aluno.
2. Leia a segunda nota do aluno.
3. Calcule a média somando as duas notas e dividindo por 2.
4. Verifique se a média é maior ou igual a 6.
5. Se for, exiba a mensagem “Aprovado”.
6. Se não for, exiba a mensagem “Reprovado”.

Fluxogramas: outra forma de representar algoritmos

Além do algoritmo narrativo, também podemos representar o mesmo algoritmo visualmente, usando símbolos padronizados conectados por setas que indicam a ordem de execução.



Terminal (início / fim)

Indica onde o algoritmo começa ou termina.



Entrada / Saída

Representa a leitura de um dado ou a exibição de um resultado.



Processamento

Representa uma ação ou cálculo, como atribuir um valor a uma variável.



Decisão

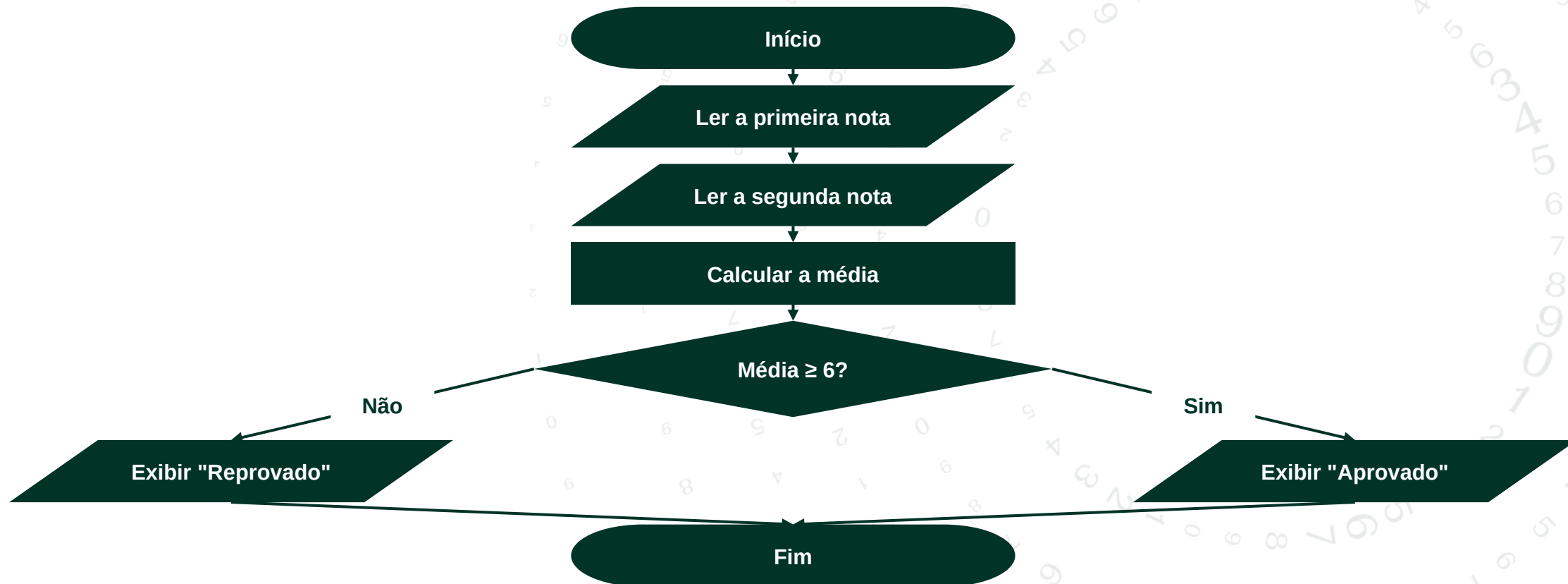
Representa uma pergunta com duas saídas possíveis, geralmente Sim e Não.

As setas sempre indicam a ordem em que os passos devem ser seguidos.



Exemplo: o mesmo algoritmo em fluxograma

Calcular a média de duas notas e informar se o aluno foi aprovado (média ≥ 6).



Constantes e variáveis

Variável

Um espaço de memória, identificado por um nome, que guarda um valor que pode mudar durante a execução do programa.

Exemplo: a idade de um usuário, que é lida e guardada em memória.

Constante

Um valor fixo, definido uma vez, que não muda durante toda a execução do programa.

Exemplo: o valor de π , ou a alíquota de um imposto fixo.



Tipos de dados

int

Números inteiros

```
idade = 20
```

double

Números com casas
decimais

```
altura = 1.75
```

string

Texto / cadeia de
caracteres

```
nome = "Ana"
```

bool

Verdadeiro ou falso

```
aprovado = true
```

char

Um único caractere

```
sexo = 'F'
```



Identificadores e boas práticas de nomenclatura

Regras (obrigatórias)

- Deve começar com letra ou underscore (_).
- Sem espaços e sem acentos.
- C# diferencia maiúsculas de minúsculas (idade ≠ Idade).
- Não pode ser uma palavra reservada da linguagem (como “class” ou “int”).

Boas práticas (recomendadas)

- Use nomes descritivos: idadeUsuario em vez de x.
- Use camelCase para variáveis: notaFinal, valorTotal.
- Nomes de constantes em maiúsculas: PI, TAXA_JUROS.

Primeiro programa em C#

Todo projeto em C# roda dentro de um programa. Na versão atual do .NET, o programa mais simples possível é só isto:

```
Console.WriteLine("Olá, mundo!");
```

- Console é a classe que representa a tela e o teclado.
- WriteLine escreve um texto na tela e pula para a linha seguinte.
- Todo comando em C# termina com ponto e vírgula (;).

Entrada de dados

Para ler o que o usuário digita, usamos `Console.ReadLine()`. Ele sempre devolve um texto (string) — mesmo que a pessoa digite um número.

```
string nome = Console.ReadLine();  
int idade = Convert.ToInt32(Console.ReadLine());  
double altura = Convert.ToDouble(Console.ReadLine());
```

Por isso, quando o dado deve ser um número, é preciso converter o texto lido com `Convert.ToInt32`, `Convert.ToDouble`, etc.

Exemplo prático: nome e idade

```
Console.Write("Qual é o seu nome? ");  
string nome = Console.ReadLine();  
Console.Write("Qual é a sua idade? ");  
int idade = Convert.ToInt32(Console.ReadLine());  
Console.WriteLine($"Olá, {nome}! Você tem {idade} anos.");
```

Repare no \$ antes das aspas: ele permite inserir variáveis diretamente dentro do texto, entre chaves {}.

Operadores aritméticos em C#

+

Soma

$$5 + 3 = 8$$

-

Subtração

$$5 - 3 = 2$$

*

Multiplicação

$$5 * 3 = 15$$

/

Divisão

$$6 / 3 = 2$$

%

Resto da divisão

$$5 \% 3 = 2$$



Cuidado com a divisão entre inteiros

Quando os dois operandos são int, o C# faz uma divisão inteira: o resultado é truncado, a parte decimal é descartada (não é arredondado).

```
int a = 7, b = 2;  
int resultadoInt = a / b;  
// resultadoInt vale 3 (a parte .5 foi descartada)  
double resultadoDouble = (double)a / b;  
// resultadoDouble vale 3.5
```

O (double) antes de "a" é um cast: converte o valor de a para double antes da divisão, forçando o resultado a manter as casas decimais.

Operador resto da divisão (%)

O operador % (chamado de módulo, ou “mod”) devolve o que sobra de uma divisão inteira — o resto, não o resultado da divisão.

$$10 \% 3 \\ = 1$$

$$8 \% 2 \\ = 0$$

$$7 \% 4 \\ = 3$$

$$15 \% 5 \\ = 0$$

Pra que serve na prática?

- Descobrir se um número é par ou ímpar: $n \% 2 == 0$ significa que é par.
- Pegar o último dígito de um número: $n \% 10$.
- Separar unidades de medida, como transformar segundos totais em minutos e segundos restantes.



Concatenação de strings com +

Para juntar textos e variáveis em uma única string, podemos usar o operador +. O C# converte números automaticamente para texto nessa operação.

```
string nome = "Ana";  
int idade = 25;  
string mensagem = "Olá, " + nome + "! Você tem " + idade + " anos.";
```

Funciona bem para poucas variáveis, mas com uma frase longa fica difícil de ler — é aí que entra a interpolação com \$, no próximo slide.

Interpolação de strings com \$

Colocando um \$ antes das aspas, podemos inserir variáveis diretamente dentro do texto, entre chaves {}. É mais legível que somar vários pedaços com +.

Com + (concatenação)

```
"Olá, " + nome + "! Você tem "  
+ idade + " anos."
```

Com \$ (interpolação)

```
 $"Olá, {nome}! Você tem {idade} anos."
```

As duas formas produzem exatamente o mesmo resultado — a escolha é sobre legibilidade.

Exercício 1

Média FECAP

Crie um programa em C# que calcule a média FECAP de um aluno. O programa deve:

1. Solicitar ao usuário as notas de NI, PI e Prova Final.
2. Calcular a média ponderada utilizando a fórmula da média da disciplina.
3. Exibir o valor da média na tela.

Fórmula

$$\text{Média} = \text{NI} * 0.20 + \text{PI} * 0.30 + \text{ProvaFinal} * 0.50$$



Exercício 2

IMC (Índice de Massa Corporal)

Crie um programa em C# que calcule o IMC de uma pessoa. O programa deve:

1. Solicitar ao usuário o peso, em quilogramas.
2. Solicitar ao usuário a altura, em metros.
3. Calcular o IMC utilizando a fórmula ao lado.
4. Exibir o valor do IMC na tela.

Fórmula

$$\text{IMC} = \text{peso} / (\text{altura} * \text{altura});$$


Exercício 3

Centena, dezena e unidade

Ler um número inteiro de 3 dígitos e apresentar a centena, a dezena e a unidade desse número.

1. Ler um número inteiro de 3 dígitos.
2. Calcular separadamente a centena, a dezena e a unidade.
3. Exibir os três valores na tela.

Exemplo para 742:

CENTENA: 7

DEZENA: 4

UNIDADE: 2

Dica: combine os operadores / (divisão inteira) e % (resto) para isolar cada dígito.



Exercício 4

Duração de uma reunião

Ler o horário de início e término de uma reunião e apresentar o tempo de duração.

1. Ler a hora e o minuto de início da reunião.
2. Ler a hora e o minuto de término da reunião.
3. Calcular a duração total, em horas e minutos.
4. Exibir a duração na tela.

Dica: transforme os dois horários em minutos totais, subtraia, e use / e % para voltar a horas e minutos.



Exercício 5

Convertendo segundos em h, min e s

Ler uma quantidade total de segundos e apresentar quantas horas, minutos e segundos ela representa.

1. Ler a quantidade total de segundos.
2. Calcular quantas horas, minutos e segundos ela representa.
3. Exibir o resultado na tela.

Exemplo para 3661 segundos:
1 hora, 1 minuto e 1 segundo

Dica: a mesma ideia dos exercícios anteriores, só que aplicando / e % duas vezes seguidas.



Recapitulando

Hoje vimos:

- O que é um algoritmo e como representá-lo: narrativo e fluxograma.
- A diferença entre algoritmo, programa e linguagem de programação.
- Constantes, variáveis, tipos de dados e boas práticas de nomenclatura.
- Primeiro contato com C#: saída (WriteLine) e entrada (ReadLine) de dados.
- Operadores aritméticos, com atenção especial ao resto da divisão (%).
- Concatenação (+) e interpolação (\$) de strings.
- Prática com os exercícios de média, IMC, dígitos e duração de reunião.

Próxima aula: Estrutura Sequencial

